

В среде Scilab встроены типы данных, которые в большинстве случаев являются основой для проведения математических расчетов – объекты.

Математические выражения в Scilab строятся на основе чисел, числовых констант, переменных, стандартных и нестандартных функций, соединенных знаками арифметических операций и круглых скобок. Кроме того, в математических выражениях могут использоваться различные спецзнаки. Вид результата зависит от установленного формата.

Число

Число – объект языка Scilab, представляющий числовые данные. Числа могут быть представлены в целом, дробном, с фиксированной и плавающей точкой, а также в экспоненциальном виде. Например:

0, 5, -6, 3.47, 0.00055, 567.9e-7, 0.89e12.

Причем числа могут быть как действительными, так и комплексными. Комплексные числа содержат действительные и мнимые части. В Scilab мнимая часть имеет множитель i , означающий корень квадратный из -1. Например:

3i, 3+5i, -i8, 0.05*% e- 0.006i

1. Константа.

Числовая константа – это предварительно определенное число (числовое значение). Числа (например, 1, -5, 3.97) являются безымянными числовыми константами.

Системные константы – это такие константы, значения которых задаются системой при загрузке, но при необходимости их можно переопределить. Имена системных констант начинаются с символа %. (таблица 1)

Таблица 1

Константа	Назначение
%i	Мнимая единица.
%pi	Число $\pi=3.1415926...$
%e	Число $e=2.141592653589793$
%eps	Условный ноль $\%eps=2.220E-16$
%inf	Значение машинной бесконечности.
ans	Переменная, хранящая результат последней операции.
%NaN	Указание на нечисловой характер данных (Not-a-Number)

2. Переменная.

Переменные – это объекты имеющие имена. Они способны хранить некоторые, разные по значению, данные.

Среди типов данных можно выделить:

- Числа
 - Целые числа
 - Вещественные числа
 - Комплексные числа
- Строки
- Логические переменные

Имена переменных (идентификаторы) задаются по следующим правилам:

- имя переменной может состоять из букв латинского алфавита (верхнего и нижнего регистра) и цифр;
- имя переменной не может начинаться с цифры, но может начинаться с символов '%', '_', '#', '!', '\$', '?';
- в имя переменной не входит пробел;
- имя переменной не должно совпадать с именами других переменных, функций, то есть должно быть уникальными;
- желательно использовать содержательные имена для обозначения переменных.

В оперативной памяти компьютера переменные занимают определенное количество байтов. Как известно эта область называется Обзорщик переменных. По умолчанию все числовые данные хранятся и выводятся на экран в формате double, что соответствует при выводе на экран format(16). Для вывода значений переменных в другом формате можно предварительно установить формат, например, format(6), в котором указывается, сколько знаков, сколько необходимо вывести. Число, введенное в скобках, учитывает количество значащих цифр, знак числа и десятичную точку. Если введенный формат достаточен для вывода значения переменной, то число с фиксированной точкой, иначе округляется и выводится в формате с плавающей точкой.

Примеры переменных:

```
--> n1=25, n2=65.33, n3=-36.4e-6, n4=9+%i*4
```

Матрицы

Матрица – это двумерный массив однотипных элементов. Можно понимать матрицу как несколько векторов-строк, записанных столбцом.

Создать матрицу в Scilab можно одним из нескольких способов:

- Матрицу можно создать из составляющих ее элементов;
- Из имеющихся векторов, упорядочив их строками или столбцами;
- Одной из специальных функций.

В общем случае синтаксическая конструкция имеет вид:

```
[x11, x12, ..., x1n; x21, x22, ..., x2n; ...; xm1, xm2, ..., xmn]
```

Создаете векторы-строки, которые отделяете точкой с запятой. Запятая в этом случае, как и с вектором, не обязательна.

```
-->A=[1 2; 3 4] // создадим матрицу из составляющих ее элементов
A =
    1.    2.
    3.    4.
```

Матрицу можно собрать из векторов

```
--> [1:5; 5:-1:1]
ans =
    1.    2.    3.    4.    5.
    5.    4.    3.    2.    1.
```

Функция

Программируемые функции призваны реализовывать некоторый алгоритм, в то время как математическая функция отражает связь между множеством аргументов и множеством значений функции.

математическую функцию можно объявить через программирование, но так как обычно ее тело состоит из одной строчки, то рациональнее всего объявлять ее через специальную функцию `deff()`. Общий синтаксис имеет вид:

```
deff('Y1,Y2...'=Fname(X1,X2,...)', ['Y1=выражение_1'; 'Y2=выражение_2;...'])
// [Y1,Y2...] – вектор возвращаемых переменных (имена назначаете сами)
// Fname – имя функции, которое вы назначаете сами
// (X1,X2,...) – список из аргументов функции
// 'Y1=выражение_1;Y2=выражение_2;...' – для каждой выходной переменной должно быть определено выражение,
// которое может зависеть от аргументов, а может и не зависеть.
```

Главное не забывать, что аргументы для функции `deff()` вы передаете в виде строк, которые затем разбиваются на лексемы и из которых среда строит программируемую функцию.

С помощью функции `funcprot()` вы можете защитить ваши функции от случайного их переопределения. Функция принимает один целочисленный аргумент:

- 0 — отключение механизма защиты;
- 1 — формирование предупреждения (используется по умолчанию);
- 2 — формирует ошибку при попытке переопределения существующей функции.

При определении функции через `deff()` она может иметь сколь угодно много инструкций, которые необходимо записывать в вектор. Запись становится неудобочитаемой и в этом случае целесообразнее использовать уже вторую

конструкцию `function...endfunction`, а также редактор `scinotes`. Общий синтаксис выглядит так:

```
function <выходные переменные>=имя_функции(аргументы)
...
инструкции
...
endfunction
```

Данная конструкция, как правило, используется для написания пользовательских программируемых функций, однако, в зависимости от предпочтений пользователя, она может использовать и вместо функции `deff()`. Использовать эту конструкцию можно в интерпретаторе, который воспринимает ее по-особому. Сначала необходимо ввести первую строку с ключевым словом `function`, именем функции, аргументами и выходными переменными. Далее можно нажать `<Enter>` и интерпретатор будет воспринимать все, что будет вводится пользователем, как инструкции для данной функции до тех пор, пока пользователь не введет ключевое слово `endfunction`. Если не было допущено синтаксических ошибок в теле функции, то она будет готова к использованию, в противном случае интерпретатор выведет ошибку и объявлять придется заново. Именно поэтому при написании больших функции следует пользоваться исключительно `scinotes`.